

Python on the IBMi- Chapter 6



As our hero presses forward, he encounters a powerful sorcerer one who twists raw data into unreadable chaos. JSON flies through the air like wild spells, and what should be a simple product list becomes a wall of confusion. But our adventurer is prepared.

In this chapter, we'll use HTML and JavaScript to dispel the mess and present our Product Master in a clean, interactive table. We'll turn backend data into a polished frontend experience breaking the sorcerer's spell one row at a time.

Disclaimer

This article will use the built-in Python HTTP Server. While this works, it is NOT production ready. While it might work for simple utilities and very low demand. It's a very simple HTTP Server that handles basic requests. The next few articles are for demonstration purposes only.

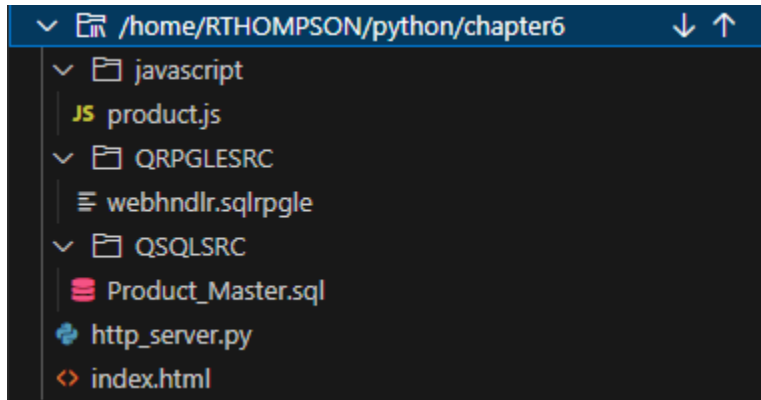
Prerequisites

- Please refer to chapter 1 for other prerequisites (<https://www.linkedin.com/pulse/python-ibmi-chapter-1-ricky-thompson-zg7te>)
- Code can be found here: (https://github.com/thompri/ibmi_python_rpg_adventures)

Objective

- Expand on Chapter 5 and display our product master in a HTML DataTable.
- Create a new folder (**/home/rthompson/python/chapter6/javascript**) and new file (**product.js**) to contain our JavaScript.

Below is what the (**/home/rthompson/python/chapter6/**) folder structure will look like with all the contents.



Let's Code

webhndlr.sqlrpgle

No code changes to our RPG program but it is provided for reference.

```
**free
Ctl-Opt option(*nodebugio:*srcstmt:*nounref) Main(Main)
dftactgrp(*no)
  text('Web Server RPG Handler Program');

// The main procedure that will listen for incoming requests
Dcl-Proc Main;
  Dcl-Pi *N;
    pMethod CHAR(10) const;
    pRequestData CHAR(32000) const;
    pResponseData CHAR(32000);
  End-Pi;

  Dcl-S vJsonString CHAR(32000);
  Dcl-S vApp VARCHAR(50);
  Dcl-S vAction VARCHAR(50);
  // Dcl-S vRequestData SQLTYPE(CLOB:500000);

  Exec Sql
    Set Option commit = *none;
```

```

If %upper(pMethod) = 'GET ' ;

    // If Request Data is blank return null
    If pRequestData = '';
        pResponseData = '{"data": null}';
        Return;
    Else;
        Exec Sql
            SELECT upper(jt.app)
                   ,upper(jt.action)
            INTO :vApp
                   ,:vAction
            FROM JSON_TABLE(trim(:pRequestData),
                           'lax $'
                           COLUMNS (
                               app    VARCHAR(50) PATH 'lax $.app',
                               action VARCHAR(50) PATH 'lax $.action'
                           )
            ) AS jt;

        If sqlcod <> 0;
            pResponseData = '{ "error": "Invalid JSON format" }';
            Return;
        EndIf;
    EndIf;

    Select;
        When vApp = 'PRODUCT' and vAction = 'GETPRODUCTLIST';
            // Call the procedure to get the Product List in JSON
format
            vJsonString = GetProductList();
            pResponseData = vJsonString;
        Other;
            pResponseData = '{ "error": "Unknown action" }';
        EndSl;
    EndIf;
End-Proc;

Dcl-Proc GetProductList;
    Dcl-Pi *N CHAR(32000);
    End-Pi;

    Dcl-S jsonString CHAR(32000);
    Dcl-S jsonStringClob SQLTYPE(CLOB:32000);

    Exec Sql

```

```

Select Cast(
  Json_Object(
    'data' Value (Json_Arrayagg(
      Json_Object(
        'ProductNumber' Value rtrim(product_number)
        , 'Description' Value Rtrim(description)
        , 'Cost' Value Cost
        , 'UnitOfMeasure' Value Rtrim(unit_of_measure)
        , 'Category' Value Rtrim(category)
      )
    ))
  ) As Clob(32000))
Into :jsonstringclob
From rthompson1.product_master;

JsonString = JsonStringClob_Data;
Return JsonString;
End-Proc;

```

I created a new folder called (**/home/rthompson/python/chapter6**). In this folder I created a HTML file named (**index.html**) with the following content. This web page on page load, will make a GET call to our Python Web Server which will then call our RPG program to get the product list and display it in a nice table. This is using jQuery to make an AJAX call and Data Tables to present the data in a nice formatted table.

index.html

```

<!DOCTYPE html>
<html>

<head>
  <meta charset="UTF-8">
  <title>Product List</title>

  <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.m
in.js"></script>
  <link href="https://cdn.datatables.net/v/dt/dt-
2.3.2/datatables.min.css" rel="stylesheet"
    integrity="sha384-
d76uxpdVr9QyCSR9vVSYd0AZeRzNUN8A4JVqUHBVXyGxZ+oOfrZVHC/1Y58mhyNg
" crossorigin="anonymous">
  <script src="https://cdn.datatables.net/v/dt/dt-
2.3.2/datatables.min.js"

```

```

        integrity="sha384-
JRUjeYWWUG0171YFugrU0ksSC6CaWn14XzwP6mNjnnDh4hfFGRyYbEXwryGwLsEp
"
        crossorigin="anonymous"></script>
<script src="javascript/product.js"></script>
</script>
</head>

<body>
    <br><br>
    <button
onclick="window.location.href='/api?action=shutdown'">Shutdown
Web Server</button>
    <button
onclick="window.location.href='/api?app=product&action=getproduc
tlist'">Get Raw Json Product List</button>

    <br><br>
    <table id="myTable" class="display table table-striped
table-bordered" style="width:100%">
        <thead>
            <tr>
                <th>Product Number</th>
                <th>Description</th>
                <th>Cost</th>
                <th>UOM</th>
                <th>Category</th>
            </tr>
        </thead>
    </table>
</body>
</html>

```

Create product.js file to contain our javascript

We will create a new folder called **(/home/rthompson/python/chapter6/javascript)**. The new file will be called **(product.js)**. Inside the **(product.js)** will be JavaScript that makes the ajax call to our web server when the page is loaded or refreshed which will in turn call our RPG program to retrieve the data. This is included in the previous (index.html) like an RPG copybook.

product.js

```

$(document).ready(function () {
    var table = $('#myTable').DataTable({
        ajax: {
            url: '/api?app=product&action=getproductlist',

```

```

        type: 'GET',
        dataSrc: 'data'
    },
    columns: [
        { 'data': 'ProductNumber' },
        { 'data': 'Description' },
        { 'data': 'Cost' },
        { 'data': 'UnitOfMeasure' },
        { 'data': 'Category' }
    ],
    pageLength: 10,
    order: [[1, 'asc']]
    });
});

```

Python Web Server Code

No code changed from Chapter 5 to Chapter 6 in our Python Web Server code but I'll repost it. I placed the file in (</home/rthompson/python/chapter6>) and called it ([http_server.py](#)).

[http_server.py](#)

```

# Import the built-in HTTP server class for handling web
requests
from http.server import HTTPServer, SimpleHTTPRequestHandler
from urllib.parse import urlparse, parse_qs

# Import argparse for command-line argument parsing
import argparse
# Import standard libraries for handling JSON
import json
import threading

# Import IBM DB2 connection tools and toolkit
import ibm_db
from itoolkit import *
from itoolkit.transport import DirectTransport

# Create a transport object that allows the toolkit to talk to
IBMi using SQL
itransport = DirectTransport()
print("Starting HTTP Server... Please wait...")

# -----
# DEFINE HTTP REQUEST HANDLER CLASS
# -----

```

```

class RPGHTTPRequestHandler(SimpleHTTPRequestHandler):
    LIBRARY = 'RTHOMPSON1' # default; will be overridden

    """
    Custom HTTP handler that serves static files and also
handles /api GET and POST requests,
    calling an RPGLE program using itoolkit.
    """

    def call_rpg(self, method, req_data):

        """
        Call the RPGLE program 'WEBSERV' in library,
        passing in the HTTP method and request data as
parameters,
        and returning the RPG response.

        Parameters:
            method (str): 'GET' or 'POST'
            req_data (str): JSON or text from the HTTP request

        Returns:
            str: response string returned from RPG program
        """

        # Initialize the toolkit object
        itool = iToolkit()

        # Add the library to the library list so the program can
be found
        itool.add(iCmd('addlible', f'ADDLIB {self.LIBRARY}'))

        # Add the RPG program call with 3 parameters:
        # - method: 10-character input
        # - request: 32000-character input
        # - response: 32000-character output (blank string
placeholder)
        itool.add(iPgm('pgmcall', 'WEBHNDLR')
                    .addParm(iData('method', '10a', method))
                    .addParm(iData('request', '32000a', req_data))
                    .addParm(iData('response', '32000a', ' '))
                    )

        # Execute the RPG call
        itool.call(itransport)

```

```

# Retrieve the output from the RPG program
result = itool.dict_out('pgmcall')

# Return the response value or fallback if missing
return result.get('response', 'No response')

# Handle HTTP GET requests to /api by calling the RPG
program
def do_GET(self):
    if self.path.startswith('/api'):

        # Parse query string into JSON string
        query = urlparse(self.path).query
        query_dict = parse_qs(query)

        # Check for shutdown command
        if query_dict.get('action', [''])[0].upper() ==
'SHUTDOWN':
            print("⚠ Shutdown requested via HTTP")

            def stop_server():
                print("▼ Shutting down server...")
                self.server.shutdown()

            threading.Thread(target=stop_server).start()

            self.send_response(200)
            self.send_header('Content-Type',
'application/json')
            self.end_headers()
            self.wfile.write(json.dumps({"status": "Server
shutting down"}).encode())
            return

        #Normal GET request processing
        query_json = json.dumps({k: v[0] for k, v in
query_dict.items()})

        request_data = query_json
        print("=== Incoming GET request ===")
        print(f"Query string: {query_json}")

        try:

            # Call the RPG program with method = 'GET'
            rpg_result = self.call_rpg('GET', request_data)
            print(f"Return Data: {rpg_result}")

```



```

        # Return the result to the client
        self.send_response(200)
        self.send_header('Content-Type',
'application/json')
        self.end_headers()
        self.wfile.write(rpg_result.encode())

    except Exception as e:
        print("=== Incoming GET request ===")
        print(f"Query string: {query_json}")

        print("=== RPG Response (raw) ===")
        print(rpg_result)

        print("ERROR during RPG call:", e)
        self.send_response(500)
        self.end_headers()
        self.wfile.write(json.dumps({"error":
str(e)}).encode())
    else:
        # Serve regular static files (like index.html, etc.)
        super().do_GET()

# -----
# START THE SERVER
# -----

if __name__ == '__main__':
    # Setup argument parser
    parser = argparse.ArgumentParser(description='Simple RPG
HTTP Server')
    parser.add_argument('--port', type=int, default=43001,
help='Port to listen on')
    parser.add_argument('--library', type=str,
default='RTHOMPSON1', help='Library to add to library list')

    args = parser.parse_args()

    # Save the library name globally
    RPGHTTPRequestHandler.LIBRARY = args.library

    # Start the HTTP server
    port = args.port
    server = HTTPServer(('0.0.0.0', port),
RPGHTTPRequestHandler)

```

```
print(f"Serving on http://localhost:{port} using library  
{args.library}")  
server.serve_forever()
```

Start the web server

Make sure the port you're using (e.g., 43001) is available on your IBMi.

From VS Code Open a PASE terminal by pressing (**Ctrl+Shift+J**) and selecting PASE and then type the following:

```
cd python/chapter6  
python3 http_server.py --port 43001 --library RTHOMPSON1
```

It will take a few seconds to start the server. You will see a message in the terminal similar to this.

```
-bash-5.2$ python3 http_server.py --port 43001 --library RTHOMPSON1  
Starting HTTP Server... Please wait...  
Serving on http://localhost:43001 using library RTHOMPSON1
```

Now open a web browser and go to your IBMi name or IP address plus the port number.

```
http://myibmi:43001/
```

You should see web page open with a list of products being returned from our RPG program in a nice little sortable table. I included the shutdown button to end the web server and a button to get the raw JSON data for convenience.

[Shutdown Web Server](#) [Get Raw Json Product List](#)

10 ▾ entries per page

Search:

Product Number	Description	Cost	UOM	Category
P0005	16GB USB Drive	9.99	EACH	Electronics
P0013	AA Batteries (4 Pack)	4.25	PACK	Electronics
P0008	Ballpoint Pens (Pack of 10)	2.99	PACK	Stationery
P0002	Bluetooth Keyboard	29.95	EACH	Electronics
P0006	Coffee Mug	5.25	EACH	Kitchen
P0009	Desk Lamp	18.75	EACH	Office
P0020	Electric Kettle	24.5	EACH	Kitchen
P0004	Laptop Backpack	45	EACH	Accessories
P0016	LED Light Bulbs (6 Pack)	12.49	PACK	Home Improvement
P0019	Mechanical Pencil Set (2 Pack)	4.75	PACK	Stationery

Showing 1 to 10 of 20 entries

« ‹ 1 2 › »

What's Next

Our hero has come far through forests of code, across rivers of integration, and past the sorcerer's illusions. But as the journey nears its end, another hidden challenge emerges from the mist: the ancient gateway of full CRUD.

We will begin to forge a complete web interface to create, read, update, and delete records in our Product Master. It will take both front-end finesse with HTML and JavaScript, and backend strength using RPG.